Buff-Bugger

A location-based, gamified platform for documenting and sharing bug finds in Boulder.

Project Report

BUGMASTERS — Team 011-5

CSCI 3308 • Spring 2026 • CU Boulder

Submitted April 24, 2026

1. Title

Buff-Bugger.

2. Team Members

Name	GitHub Username
Molly Dembo	*PigeonTheSloth*
Trent Clatanoff	*trentclat*
Benjamin Davison	*Ben-Dav*
Amrit Hari	*Amrit-Hari*

3. Project Description

Buff-Bugger is a location-based web application for discovering, documenting, and sharing insect and arthropod finds around Boulder, Colorado. Unlike existing biodiversity tools such as iNaturalist and Seek — which are optimized for scientific data logging — Buff-Bugger frames bug observation as a game. Users photograph a bug they encounter, upload it with its location and a short description, and the find appears as a marker on a shared map of the Boulder area. Over time, users build a personal collection of finds and compete on a leaderboard with other members of the community.

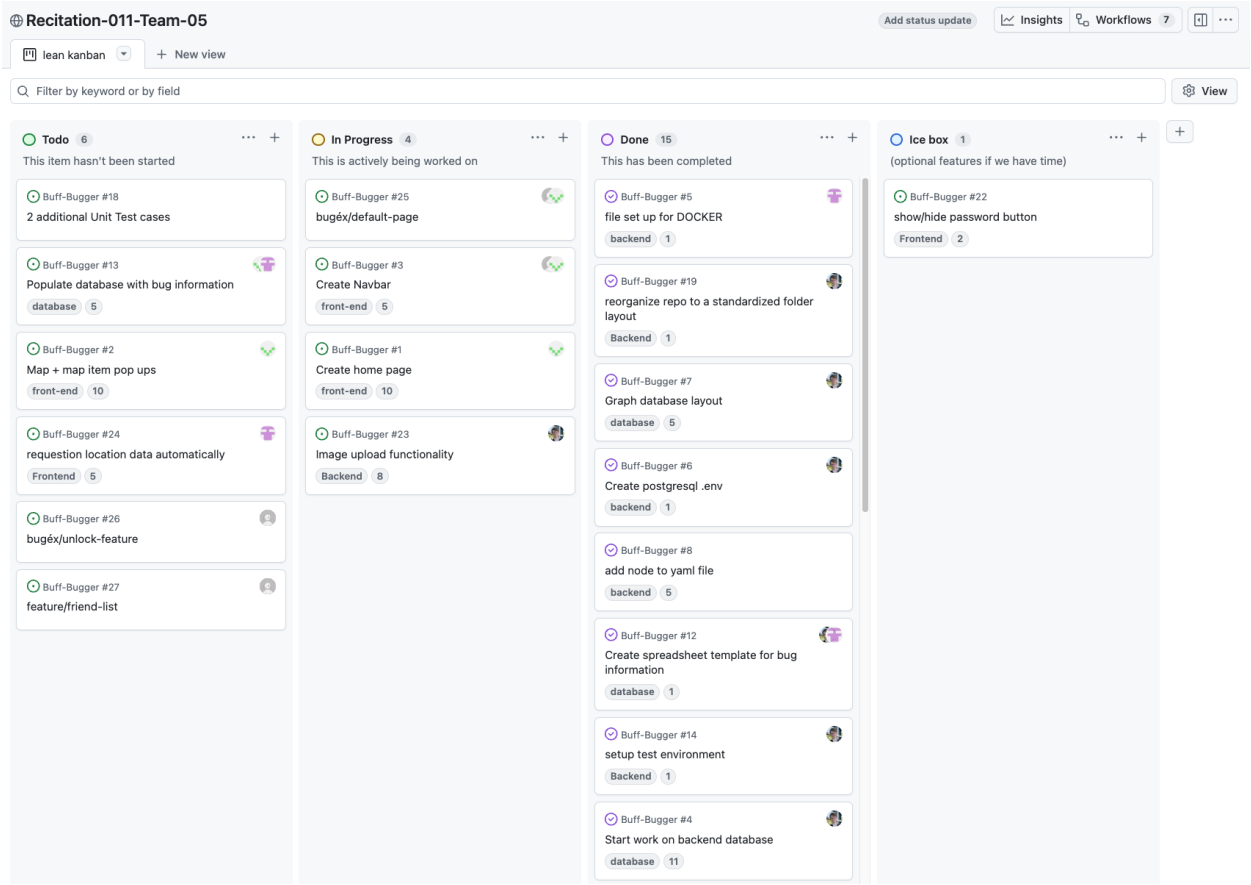
The platform runs on a Node.js and Express server backed by a PostgreSQL database, with Handlebars templating for the front end and Google Maps JavaScript API for map rendering. The entire stack is containerized with Docker Compose for reproducibility, hosted on Render for public access. Authentication is handled with bcryptjs and session-based login, and uploads are handled via multer. The project represents a deliberate attempt to make a category of activity — local biodiversity observation — feel more like an exploration game and less like a data entry task.

4. Project Tracker

Link to project board:

project board

Screenshot:



The board tracks user stories as issues, organized across columns representing stages of the development lifecycle. Stories are linked to epics and assigned to individual team members, with status updated throughout the sprint.

5. Demo Video

Link

The demo follows the primary user journey: landing on the home map with existing bug markers, registering and logging in, submitting a new bug find with a photo and location, viewing the new marker on the map, and exploring the leaderboard and AI-assisted identification features.

6. Version Control (Git Repository)

[Respository link]

The repository is organized per the course-specified structure:

- TeamMeetingLogs/ — minutes of weekly TA meetings, one file per week
- MilestoneSubmissions/ — weekly release notes and submissions
- ProjectSourceCode/ — application code, Docker configuration, tests
- README.md — project overview and run instructions

Commit history reflects continuous development across all four team members, with feature branches merged into main via pull request after peer review.

7. Individual Contributions

Molly Dembo

I worked on a mixture of front end and back-end features. I was responsible for the implementation of the map API integration that draws from the DB to render markers on the map. I also worked on initial versions of login and registration, including password security, that were later elaborated on by Ben. I created the functionality for the leaderboard page worked, and made the navigation bar as well. I also worked on styling on the front end. Overall, I helped with a variety of problems and generally assisted where I was needed. I also was responsible for doing Lab 12 and setting up the render server hosting.

Trent Clatanoff

I owned mostly back-end and full-stack delivery of two core features for the Buff-Bugger platform. I built the **"Bug-i-dex"** interface and functionality end-to-end, which is a Pokedex-style species tracker with per-user caught/uncaught states, new `bug_info` database schema, route logic, and themed CSS. I also integrated an **AI chatbot** via the Anthropic API for the genAI usage extra credit portion of the project requirements. Also included is a /api/chat server route that injects each user's caught species into the system prompt, so you can chat about the actual state of your bug-i-dex.

More generic contributions include initial project infrastructure: Docker-compose SQL mounts, init-script load ordering, layout shell, home render, and demo video.

Benjamin Davison

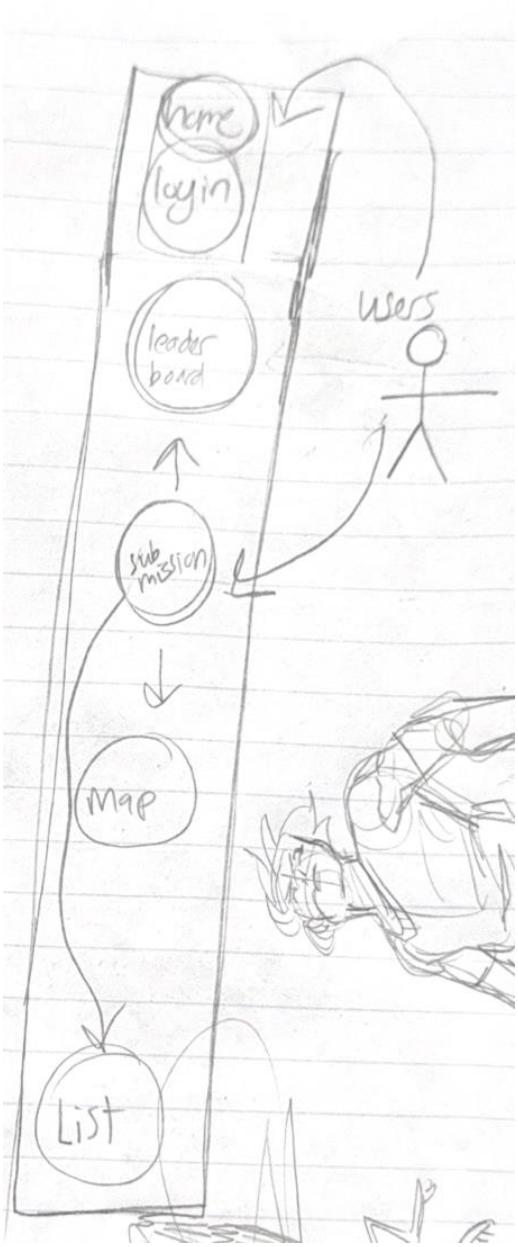
I worked mostly on the backend with mixed frontend for the user information handling systems-allowing the end users to login and registration along with post submissions systems, with per-user caught/uncaught states, and route logic. I also owned the database, having designed and implemented most of the schema. More generic contributions include project structure and conformity, the Docker-compose, and front-end layout streamlining. I also owned the database, having designed and implemented most of the schema and backend interactions.

Amrit Hari

I contributed across infrastructure, data setup, frontend scaffolding, and documentation for Buff-Bugger. I helped set up the Docker/PostgreSQL environment and working app foundation, added

homepage Handlebars code, expanded the bug dataset with Boulder bug seed data, and improved bug info lookup/indexing in the SQL layer. I also worked on project-facing documentation by updating the README with run instructions and project intro, and adding a contributing guide and design system. My role focused on helping make the project runnable, structured, and easier to present and maintain.

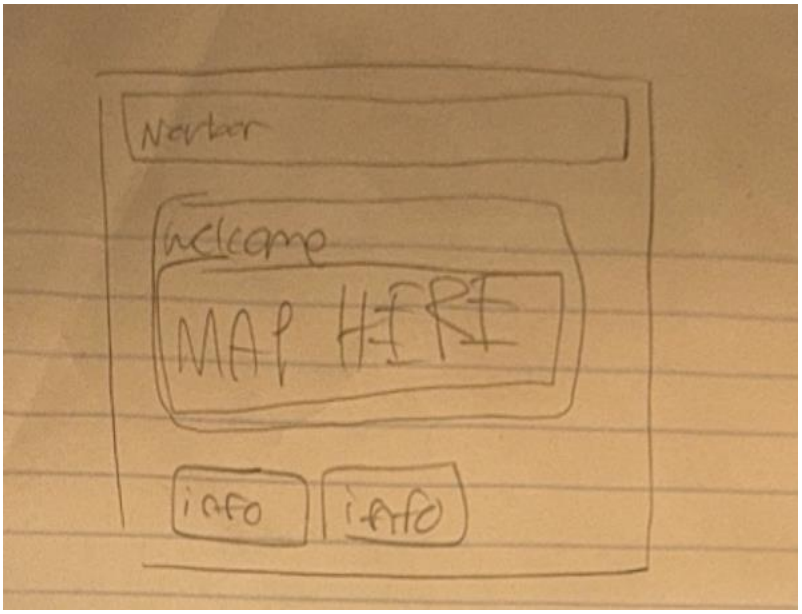
8. Use Case Diagram



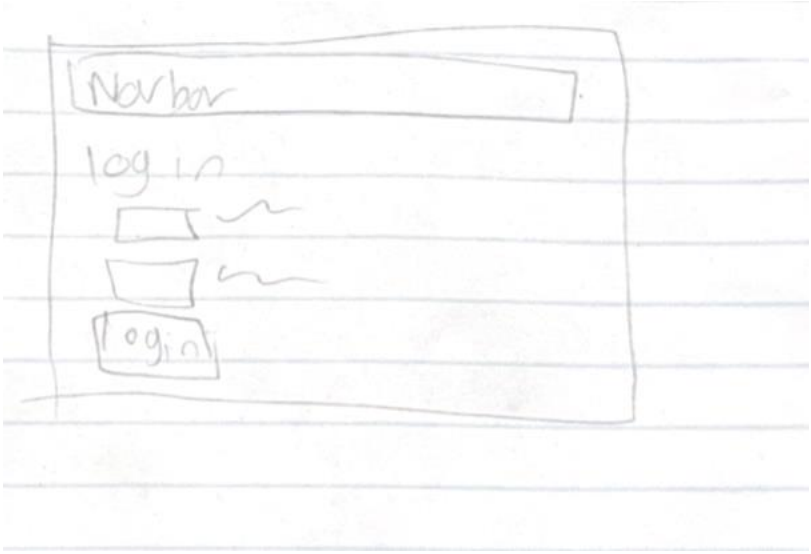
The primary actor is a registered user, whose use cases include: registering an account, logging in and out, viewing the Boulder-area bug map, submitting a new bug find (with photo, location, and description), viewing other users' finds via the map, and consulting the leaderboard to see relative standings within the community. A secondary actor is the unauthenticated visitor, whose available actions are limited to viewing the landing page and registering or logging in.

9. Wireframes

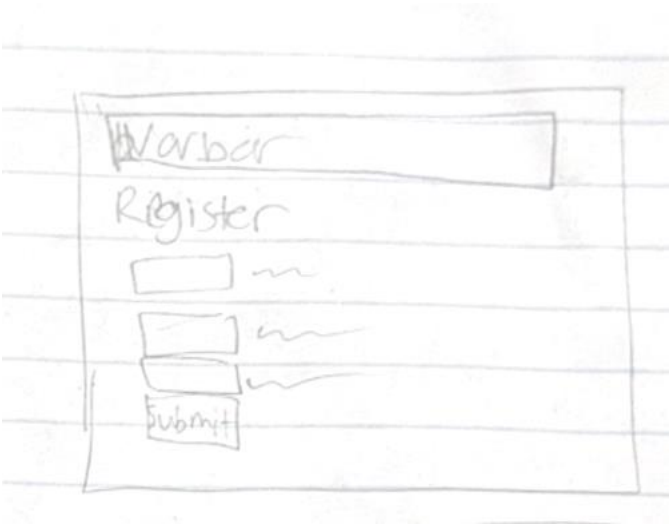
Home page (map view)



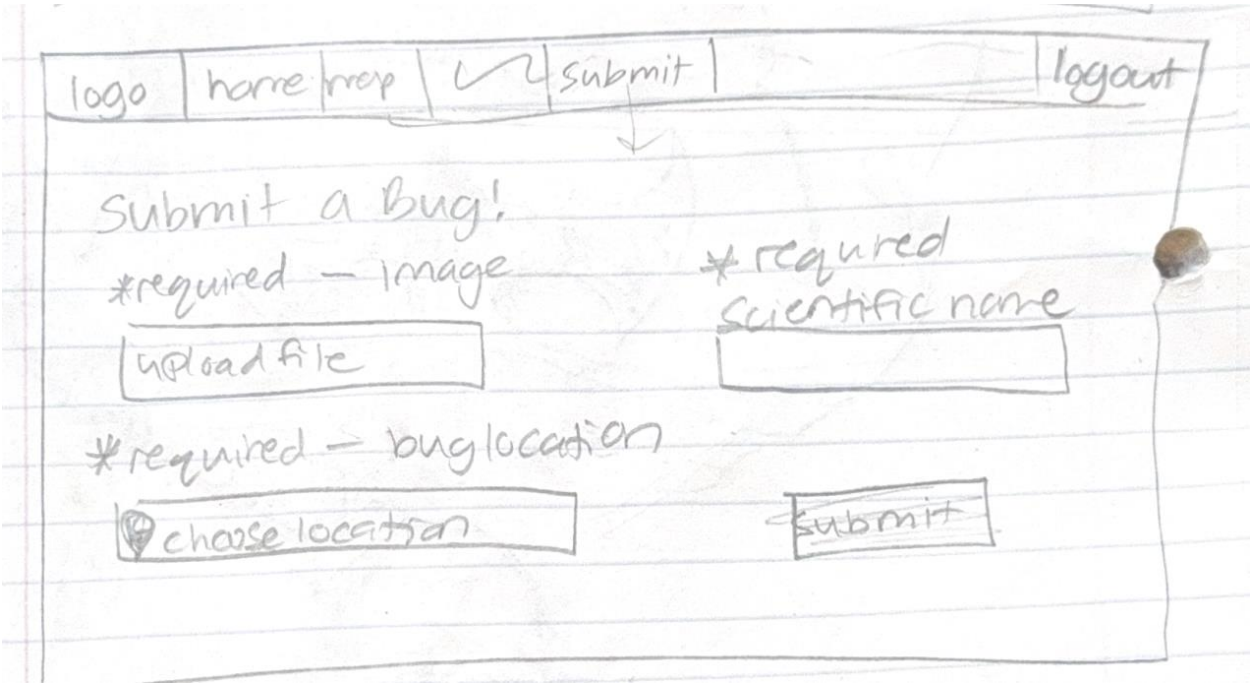
Login page



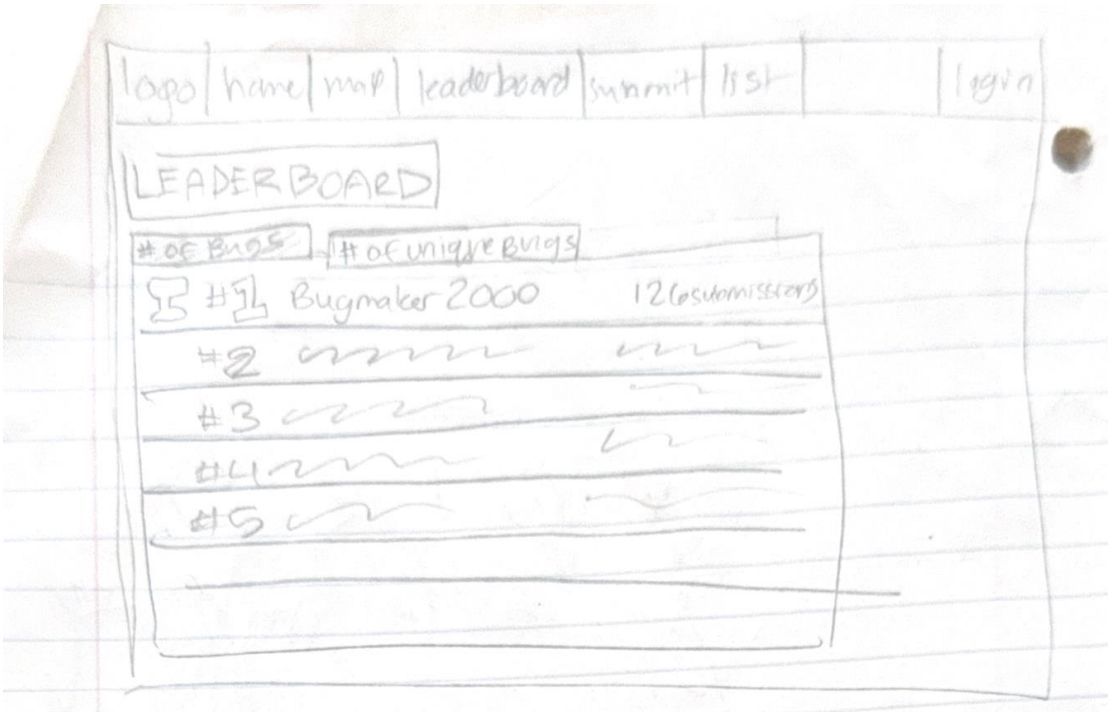
Registration page



Bug submission page



Leaderboard page



Bugindex page

10. User Acceptance Test Results

We conducted user acceptance testing with a small panel of test users drawn from classmates and peers outside the project team. Each user was asked to complete four core scenarios while the team observed and took notes without intervening.

Scenario 1: Account Registration

Expected behavior: User navigates to the registration page, enters a username, password, password confirmation, and a valid email, and submits. User is then logged in.

Observation:

User took a while to make a password that passed the validity check when registering because a '-' didn't count as a special character. The user logged in easily after registration and had no problems.

Changes made based on feedback:

Made – a valid special character

Scenario 2: Login

Expected behavior: User enters username and password on the login page. On success, session is established and protected pages become accessible.

Observation:

User logged in intuitively with no problems.

Changes made based on feedback:

None.

Scenario 3: Bug Post Creation

Expected behavior: Logged-in user shares their location, shares their location, fills out the submission fields, and submits.

Observation: User seemed to understand the submission fields easily. They understood that their item got added to the map.

Changes made based on feedback:

None.

Scenario 4: Map Interaction

Expected behavior: User views the Boulder-centered map on the home page, clicks a marker, and sees an info window with common name, genus, and color.

Observation:

User clicked on marker spontaneously as expected and zoomed in to investigate other markers on the map.

Changes made based on feedback:

none

11. Deployment

Deployed application URL:

https://buff-bugger.onrender.com/home

Running Locally

The application can be run locally using Docker Compose. Prerequisites: Docker Desktop (or Docker Engine), Node.js (optional, for running tests outside the container), and a .env file matching the .env.example template in the repository.

- **Step 1:** Clone the repository and navigate into ProjectSourceCode/.
- **Step 2:** Copy .env.example to .env and populate the required environment variables.
- **Step 3:** Run docker-compose up from the ProjectSourceCode/ directory.
- **Step 4:** Open http://localhost:3000 in a browser.

Running Tests

Automated Mocha/Chai tests are included for the server endpoints. To run:

- docker-compose run web npm run test

The test suite covers the welcome endpoint, positive and negative registration flows, and the /api/bugs data endpoint used to populate map markers.

12. Extra credit

Our team pursues all four extra credit categories from the course guide.

12.1. Innovative Problem Selection

Existing biodiversity tools (iNaturalist, Seek) are optimized for scientific data logging and feel closer to filling out a form than going on an expedition. Buff-Bugger reframes the same activity as a hyper-local game centered on social competition rather than global data contribution. The "Why Buff-Bugger" section at the top of the README expands this case in detail.

12.2. Creativity of Website UI/UX

The repository includes a DESIGN.md document specifying the project's full design system: a forest/moss/amber color palette, Georgia + system-sans typography pairing, component patterns for cards and forms, voice and tone guidelines, and an accessibility baseline. The Bug-i-dex page applies this palette through themed CSS.

12.3. Responsible GenAI Integration

The Bug-i-dex page includes an Anthropic Claude-powered chatbot (/api/chat) that lets users ask questions about their own caught species. The user's caught/uncaught state is fetched server-side and injected into the assistant's system prompt, so the chatbot operates on the user's actual data rather than hallucinated species. Implementation details and architectural choices are documented in ec-chatbot.md. A separate writeup on responsible GenAI usage is in genAI_usage_extra_credit.txt. Implementation owner: Trent Clatanoff.

12.4. Professional Organization & Documentation of Repository

The repository includes a README.md covering all course-required sections (description, contributors, tech stack, prerequisites, run instructions, test instructions, deployed link), a CONTRIBUTING.md documenting branch conventions, commit message format, and the pull request review process, a DESIGN.md design system, an .env.example template for environment variables, and a LICENSE file. The repository follows a feature-branch workflow with peer-reviewed pull requests, and weekly meeting minutes and milestone release notes are maintained in their respective folders.